
flownexus Documentation

Release v0.1.1-31-g2ec636e

Jonas Remmert, Akarshan Kapoor

Jan 29, 2026

DOCUMENTATION

Documentation	i
1 Overview	2
1.1 Features	2
1.2 Key Components	2
1.2.1 IoT Devices	2
1.2.2 LwM2M Server	2
2 Architecture	4
2.1 Django	4
2.1.1 Build and Run	4
2.1.2 Database Model	5
2.2 Leshan	6
2.2.1 Build and Run	6
2.2.2 Leshans Role in flownexus	6
2.2.3 LwM2M Observe	7
2.3 Communication, Interfaces	7
2.3.1 Data Flow: Endpoint -> Backend	7
2.3.2 Data Flow: Backend -> Endpoint	9
2.4 Over the Air Updates	10
2.4.1 Update process of a Firmware	10
3 Application Guide	12
3.1 Endpoint Interactions	12
3.1.1 Read	12
3.1.2 Write	12
3.1.3 Execute	12
3.1.4 Observe	12
3.2 Events	12
3.2.1 Events via Composite Observe	13
3.3 Queue Mode	13
3.3.1 eDRX Settings	13
3.4 Firmware Setup	13
3.4.1 LwM2M Version	13
3.4.2 LwM2M Transport Format	14
3.4.3 LwM2M SenML Format	14
4 Build and Deploy	15
4.1 Build Setup	15
4.2 Container Environment	16
4.3 Setup a Virtual Server	16
4.3.1 CA and self-signed Certificate	16
4.3.2 Nginx as Reverse Proxy	17
4.3.3 Test the Download Server	19
4.3.4 Start flownexus	19

4.3.5	Security Considerations	19
5	Simulation	21
5.1	Simulation Backends	21
5.2	Simulation Dispatcher (simulate.py)	21
5.3	Mock Simulation	21
5.3.1	Features	22
5.3.2	Running the Mock Simulation	22
5.4	Zephyr Simulation	22
5.4.1	Prerequisites	22
5.4.2	Running the Zephyr Simulation	22
5.4.3	Manual Control	22
5.5	Configuration (YAML)	23
5.6	Integration Testing	23
5.7	External Resources	23
6	Documentation	24
6.1	Build the documentation	24
7	Glossary	25
7.1	Additional Terms and Definitions	25
8	Internal Django	26
9	Weblog	27
9.1	Release v0.1.0	27
9.2	Update Documentation & Add Weblog	27
9.3	Project Starts	28
	Index	29
	Index	29

Introduction

Welcome to flownexus, your comprehensive solution for small to medium scale IoT deployments. This platform is designed to streamline the setup, configuration, and use of IoT devices, making it easy to manage your IoT ecosystem.

This project aims to provide a minimal and standalone Open Source IoT platform. The focus is on systems with a deployment of up to a few thousand devices; it does not aim to be scalable to millions of devices. The goal is to build a system that receives, for example, regular temperature samples from remote IoT devices. These temperature samples will be sent to a backend and stored in a database. The data is then visualized in a web application.

Our vision for flownexus is to create a user-friendly, efficient, and robust Open Source platform for IoT management. We aim to simplify the complexities of IoT deployments, making them accessible and manageable for businesses of all sizes.

OVERVIEW

This documentation describes a local server-based IoT system that leverages the Lightweight Machine to Machine (LwM2M) protocol to communicate between IoT devices running Zephyr OS and a backend server using Django. The system does not depend on external cloud services and is designed to operate fully within a local environment.

1.1 Features

- No dependencies of external services like AWS, MQTT brokers or similar. The system has to be able to run in a local environment.
- The main focus is the LwM2M protocol and the communication between Zephyr and the server.
- The system should show how to add data to a database and visualize it in a web application.
- The web application has to support a secure login and basic user management.
- Certificate based authentication and encryption.
- Server to read and observe resources from IoT Device.
- Server to write resources to IoT Devices.
- OTA Update.
- Receive logs from IoT Devices.

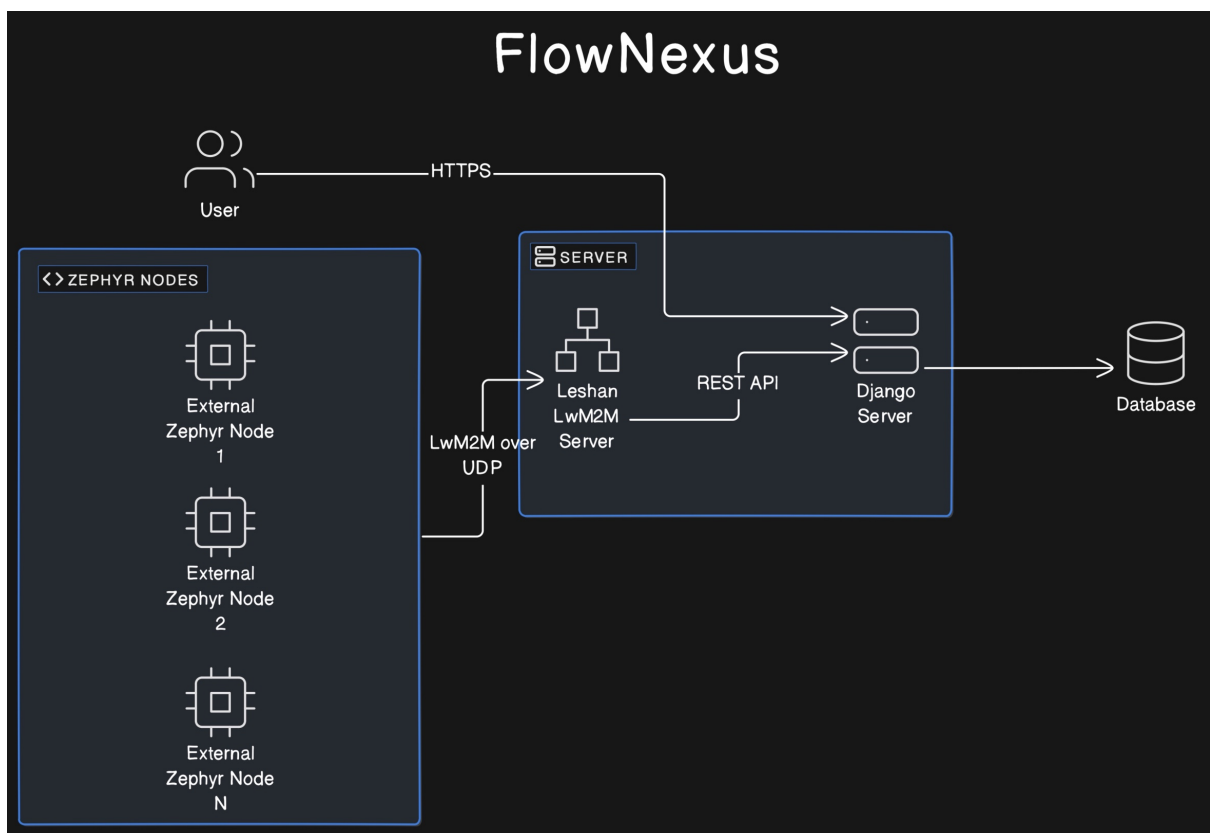
1.2 Key Components

1.2.1 IoT Devices

These are typically resource-constrained devices (limited energy and storage capabilities) that run Zephyr OS. They communicate with the server using the UDP protocol to ensure constant connectivity, even in low power states. These devices are programmed to handle tasks like temperature sensing, location tracking, or any other telemetry based on Zephyr applications. They communicate via the UDP protocol to maintain a continuous link without needing frequent reconnections. The IoT device has to support LwM2M. Mainly systems that run Zephyr should be compatible. The application can e.g. be used with a dedicated nRF9160 device or via a simulation like `native_sim` or in Renode. The nRF9160 is a low power LTE-M and NB-IoT SoC that runs Zephyr OS. UDP is used as transport protocol as it allows to keep devices connected even when the device is sleeping for extended periods (TCP would require a new connection setup typically after a few minutes)

1.2.2 LwM2M Server

The LwM2M server plays a critical role in managing communications with IoT devices. It handles data transmission, device monitoring, and control commands, ensuring that devices can report their telemetry data, receive configuration updates, and execute commands sent by the server. The server's efficient management of these tasks is essential for maintaining reliable communication with numerous IoT devices, especially those with limited power and processing resources.



ARCHITECTURE

This section describes the architecture of individual components and how they interact with each other.

2.1 Django

The Django server hosts the REST API, manages the database, and provides a web interface for data visualization and user management. This server acts as the backend infrastructure that supports the entire IoT ecosystem by:

- **REST API:** Facilitates communication between the IoT devices and the server. The API endpoints handle requests for data uploads from devices, send control commands, and provide access to stored telemetry data.
- **Database Management:** Stores all the telemetry data, device information, user accounts, and configuration settings. The database ensures that data is organized, searchable, and retrievable in an efficient manner.
- **Web Interface:** Offers a user-friendly interface for data visualization and management. Through this interface, users can monitor device status, visualize telemetry data in real-time, manage user accounts, and configure device settings.

2.1.1 Build and Run

The Django server can also run locally, without the need of a container. Make sure to create a virtual environment and install the requirements:

```
host:~$ source venv/bin/activate
host:~$ cd workspace/ flownexus/server/django
host:~/workspace/ flownexus/server/django$ pip install -r requirements.txt
host:~/workspace/ flownexus/server/django$ ./django_start.sh
```

The Django server should now be up and running under the following URL: <http://localhost:8000/admin>. The admin login is `admin` and the password

Run Unit Tests

There are unit tests, that test the deserializer, which parses the Json payload from the ReST API. You can run the unit tests with the following command:

```
host:~/workspace/ flownexus/server/django$ python manage.py test sensordata
Found 2 test(s).
Creating test database for alias 'default'...
-----
Ran 2 tests in 0.008s

OK
Destroying test database for alias 'default'...
```

2.1.2 Database Model

The database model is the core of the Django server. It aims to store information according to the LwM2M resource model. The advantage is that data can be stored in a generic way and the server can be extended with new resources without changing the database schema.

The server application logic only has to handle higher level events. Those events are situations where multiple resources are associated (e.g. Temperature, Pressure, Humidity, Acceleration). Those multiple resources are linked in the event, together with a timestamp. The event itself is represented by the database model in a generic way, several custom event types can be created by the application logic.

An Entity Relationship Diagram (ERD) is a visual representation of the database schema. It is automatically generated from the Django models. `sensordata` is the Django app that contains the application logic.

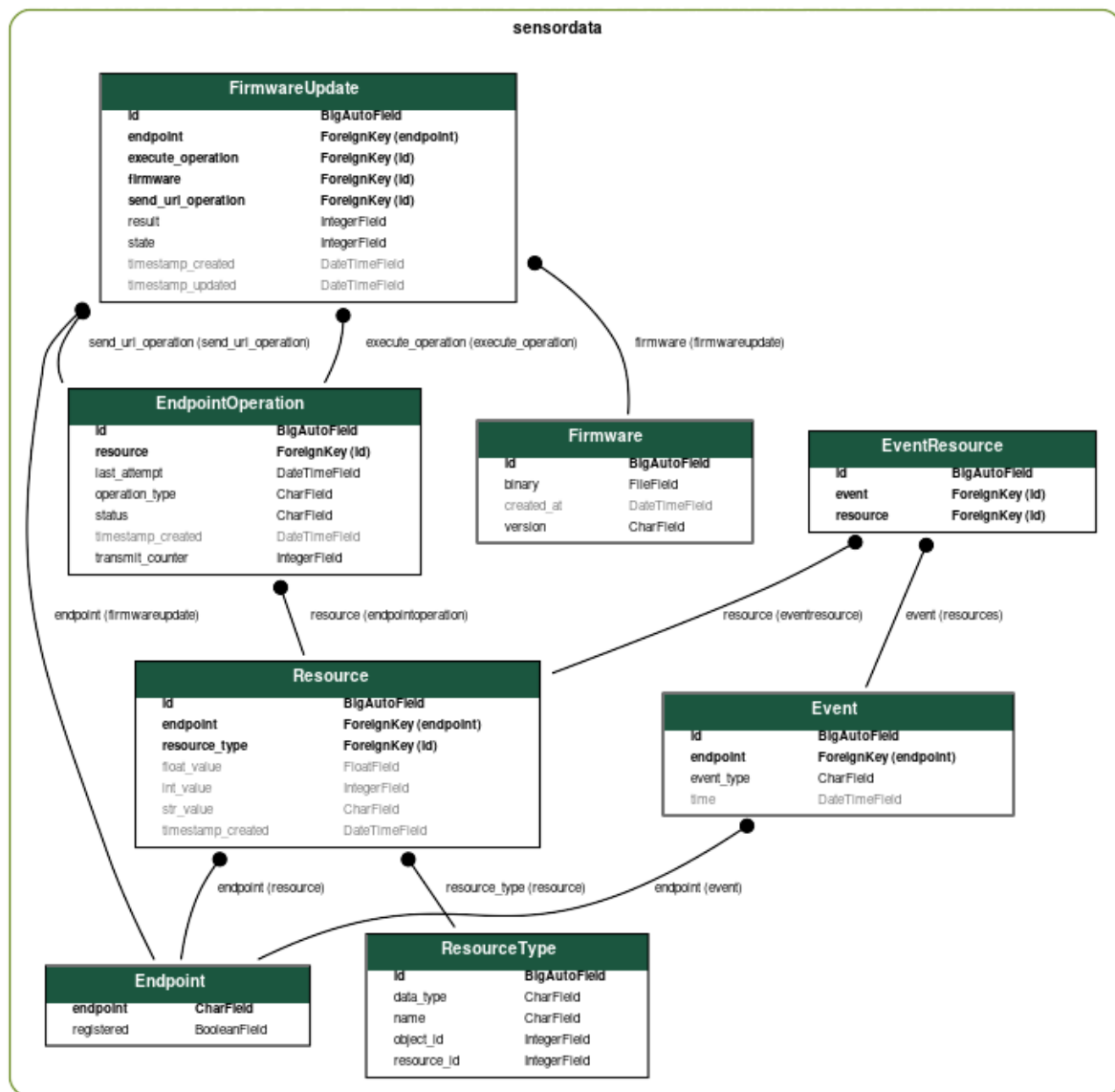


Fig. 1: Entity Relationship Diagram generated from Django models

Endpoint

IoT devices using the LwM2M protocol in the network, identifiable by a unique name, e.g. `urn:imei:123456789012345`.

ResourceType

Defines resource data points comprehensively, annotating each with a unique object-resource ID combination, a descriptive name, and specifying the expected data type.

Resource

A specific piece of data or functionality within an LwM2M Object. Resources represent attributes or actions and are identified by Resource IDs within an Object.

In the database Model a Resource represents an individual data value from one IoT device, annotated with timestamps, applicable data types, and linked to both the device and resource type for which the data is relevant.

Event

A collection for significant occurrences reported by endpoints. Events and Resources are linked to each other via EventResource table.

EventResource

Acts as a junction table forming a many-to-many relationship between events and their constituent resources, enabling flexible association without direct modification to the core events or resources tables.

EndpointOperation

Represents actionable commands or processes targeted at endpoints, tracking the operation type, status, and scheduling through timestamps, also detailing the transmission attempts and last action.

Firmware

Stores metadata about firmware binaries that are available for devices to download and install. Each record includes a version identifier, the name of the file, a URL from where the device can retrieve the firmware, and timestamps for tracking when each firmware record was created and last updated.

FirmwareUpdate

keeps track of the execution of firmware updates for each endpoint. It adds references to the two required resources from server to endpoint (Send URI, execute Update). Furthermore it adds a field for the State and the Result of an update.

2.2 Leshan

2.2.1 Build and Run

The Leshan server can also run locally, without the need of a container.

```
host:~$ sudo apt update
host:~$ sudo apt install openjdk-17-jdk maven
host:~$ source venv/bin/activate
host:~$ cd workspace/flownexus/server/leshan
host:~/workspace/flownexus/server/leshan$ ./leshan_build_run.sh
```

The Leshan server should now be up and running under the following URL: <http://localhost:8080>.

2.2.2 Leshans Role in flownexus

Leshan is considered a production ready LwM2M server and is used in many commercial products. It is used as an LwM2M server in flownexus. It is responsible for registering and managing [endpoint](#). There is no application specific logic implemented in Leshan related Java code within flownexus. The goal is to have all application logic within the Django application.

Leshan can be seen as a proxy between the LwM2M endpoints Django. There is no direct communication between Django and endpoints.

2.2.3 LwM2M Observe

LwM2M observe

The *LwM2M observe* operation allows an LwM2M Server to monitor changes to specific Resources, Resources within an Object Instance, or all Object Instances of an Object on an LwM2M Client. An endpoint must remember these observation requests until re-registration.

Single and CompositeObserve

Leshan supports two types of observe requests: Single and CompositeObserve. SingleObserve is a simple observe request for a single resource. CompositeObserve is a more complex observe request that subscribes to multiple *Resources* of one *Object*.

Once an endpoint registers to Leshan, Leshan will initiate an observe request to selected resources. With every new registration of an endpoint, the observe requests are re-initiated.

Note

The observe initiation will move from Leshan to Django with future versions. Django is informed about new registrations and will then initiate the observe requests. This allows a more centralized approach and better control over the observe requests via the database model.

For testing, the Leshan server currently initiates the following two observe requests:

- SingleObserve (Temperature Sensor Value): {3303, 0, 5700}
- CompositeObject (Custom Object Id): {10300}

2.3 Communication, Interfaces

The communication between IoT devices and Leshan is specified by the OMA LwM2M standard:

- [LwM2M core specification v1.1.1](#)
- [LwM2M transport binding v1.1.1](#)

The standard describes how the LwM2M server (Leshan) works, however, it does not describe how to connect a backend server to Leshan. The backend is responsible for storing the data in a database and implementing application logic. A frontend can access the data in the database and visualize outward facing user interfaces. Leshan acts as a gateway between Endpoints and the backend. There should be no application specific logic implemented in Leshan.

In order to communicate and exchange data, both components (Leshan LwM2M Server and Django) post data to each other's ReST APIs. Communication is typically triggered by IoT devices sending data or the user/application requesting data from devices.

2.3.1 Data Flow: Endpoint -> Backend

All communication from Endpoints to the server flows through Leshan. Leshan interprets this data according to the LwM2M protocol and generates a ReST API call to the backend server. The backend server then stores the data in the database.

Django hosts the ReST API that Leshan posts to. Serializers, a part of Django, deserialize the incoming data and store it in the database according to the database model.

Leshan Data Format

There are two types of data that Leshan sends to the backend, single resource and composite resource format. The two ReST API endpoints that Leshan posts to are available under the following URLs:

- `/leshan_api/resource/single`: single resource format.
- `/leshan_api/resource/composite`: composite resource format.

For more details, please check the [Internal Django API documentation](#).

Listing 1: Single Resource Format (3303/0/5700)

```
{
  "ep": "qemu_x86",
  "obj_id": 3303,
  "val": {
    "kind": "singleResource",
    "id": 5700,
    "type": "FLOAT",
    "value": "24.899181214836236"
  }
}
```

Listing 2: Composite Resource Format (3/0/0..17)

```
{
  "ep" : "qemu_x86",
  "val" : {
    "instances" : [ {
      "kind" : "instance",
      "resources" : [ {
        "kind" : "singleResource",
        "id" : 0,
        "type" : "STRING",
        "value" : "Zephyr"
      }, {
        "kind" : "multiResource",
        "values" : {
          "0" : "1",
          "1" : "5"
        },
      },
    ],
    "kind" : "singleResource",
    "id" : 17,
    "type" : "STRING",
    "value" : "qemu_x86"
  } ],
  "id" : 0
} ],
"kind" : "obj",
"id" : 3
}
```

The marked lines in the composite resource format show where Object ID, Instance ID and Resource ID are located. A composite resource format can consist of multiple Object IDs.

Warning

Currently multiResources are not supported and will be ignored. MultiResource datatypes are e.g. used for Voltage range (min, max).

Registration Updates

Leshan maintains registrations of endpoints. An endpoint can be registered or unregistered. To maintain its registration, an endpoint must send an update to Leshan regularly. If an endpoint does not send an update within the specified duration, it is considered offline and will be unregistered by Leshan.

Those registration events are encapsulated into an LwM2M Object and sent to the backend server. The backend server stores the registration events in the database. This allows to use the generic database model for the registration events as well. All received registration events are stored in the database and can be used for statistics.

All registration events are maintained in the custom LwM2M Object ID 10240:

- 10240/0/0: Endpoint **registered** to Leshan.
- 10240/0/1: Endpoint **unregistered** from Leshan.
- 10240/0/2: Endpoint **updated** its registration.

2.3.2 Data Flow: Backend -> Endpoint

Endpoints often operate in queue mode, meaning they are not always online. The LwM2M Server is aware of the current status of a device (Online/Offline) and communicates this status to the backend server. Leshan does not queue pending data that should be sent to the device when it comes online. The backend server must handle this by itself so it has to have a representation of the current status of each device as well as the data to be send. The resource table **EndpointOperation** is used to store pending operations that should be sent to the endpoint while it is online.

Once an endpoint updates it's registration (LwM2M Update Operation) Leshan notifies the backend. The backend checks the **EndpointOperation** table for pending operations and sends them to the device by posting to the Leshan hosted ReST API. Leshan keeps the post call open until the device acknowledges the operation or a timeout is generated. Endpoints can be slow to respond (several Seconds), so the backend has to handle the ReST API call in an asynchronous manner. By only sending data to endpoints while they are online, the backend can be sure that the ReST API calls are not open for a long time.

Asynchronous Communication

Given that endpoints are comparably slow to respond, handling communication asynchronously is essential for efficient operation. This can be effectively managed using Celery, a distributed task queue. When Leshan notifies the backend of an endpoint status update, Celery can be used to handle the long-running API calls, ensuring that the backend remains responsive and scalable. As the backend communicates with many endpoints simultaneously, an efficient queuing mechanism is essential to ensure that the system remains responsive and scalable.

Before the backend executes the API call, it updates the **endpointOperation** status to **SENDING**, indicating an ongoing operation.

Once the API call is complete the database will be updated with the result (e.g. **CONFIRMED**, **FAILED**, **QUEUED**) depending on the result of the request. The **FAILED** status is assigned after 3 attempts. Retransmissions are triggered when the endpoint updates it's registration the next time.

Example Communication

The following example shows how the backend server can send a firmware download link resource **Package URI 5/0/1** to an endpoint:

1. User creates new **EndpointOperation**: resource path 5/0/1, value **https://url.com/fw.bin**.

2. Backend checks endpoint online status.
3. If endpoint is offline, no further action is taken right away.
4. Endoint comes online, Leshan sends update to the backend.
5. Backend checks `EndpointOperation` table for pending operations for the endpoint.
6. Finds pending operation, send resource to endpoint via the Leshan ReST API.
7. Pending operation is marked `completed` if the endpoint acknowledges the operation.

2.4 Over the Air Updates

Over the Air (OTA) updates are a way to update the firmware of a device remotely. This is a mandatory feature for any IoT device. The device must be able to update its firmware without any physical user intervention.

flownexus implements OTA updates according to the LwM2M protocol. The LwM2M protocol specifies how to update the firmware of a device. The OTA process is implemented by sending a link to a firmware via LwM2M object 5/0/1 - Package URI. This method is called Pull via HTTP(s) and is faster, compared to a transfer via CoAP. Updates via CoAP blockwise transfer to download the firmware are not supported.

A https server is usually be hosted under a subdomain. The default is `https://fw.flownexus.org`. This makes it easy to use a self-signed certificate for the https server. For details, check chapter *Setup a Virtual Server*.

More information about a possible firmware implementation can be found in the `lwm2m_client` firmware sample at `flownexus/simulation/lwm2m_client/src/firmware_update.c`.

Note

Firmware Updates are implemented in the LwM2M server and the example firmware. A set up a PKI with self-signed certificates is required for the server to communicate with the device via https. This setup will be described in upcoming releases shortly.

2.4.1 Update process of a Firmware

The update process is described in detail in the [LwM2M core specification v1.1.1](#) Chapter *E.6 LwM2M Object: Firmware Update*. This chapter summarizes the update process:

1. flownexus sends a link to the firmware to the device via LwM2M object 5/0/1 - Package URI. It is a relative link to the firmware on the server, e.g. `/binaries/v0.1.0.bin`. The host `https://fw.flownexus.org` is fixed in firmware.
2. The device downloads the firmware from the server via http(s) and sets the state to `STATE_DOWNLOADING`.
3. The device sets the state to `STATE_DOWNLOADED` after the download is complete.
4. flownexus executes the command 5/0/2 - Update to the device.
5. The device sets the state to `STATE_UPDATING` upon receiving the command.
6. The device updates the firmware, usually by performing a reboot.
7. The device registers and sends the firwmare version to flownexus via resource `Firmware Version - 3/0/3`. Depending if the update was successful or not, flownexus sets the result and state of the update in the database accordingly.

This process ensures that errors during the update process are handled and communicated to the backend.

States of a OTA Update

There are 4 device states during an OTA update. flownexus tracks the state of the device during the update process in the database (FirmwareUpdate table). Once the update is completed (failed or successful), the state is set always set to `STATE_IDLE`.

Table 1: Device States during an OTA Update

State	Description
<code>STATE_IDLE</code>	device is idle
<code>STATE_DOWNLOADING</code>	device is downloading
<code>STATE_DOWNLOADED</code>	download is complete
<code>STATE_UPDATING</code>	device is updating

Result Codes for an OTA Update

For every OTA update, a result is generated after the update failed or successful. There can be only one active OTA update at a time for each client. An active OTA is indicated by the result `RESULT_DEFAULT`. The result for each OTA update is stored in the database (FirmwareUpdate table).

Table 2: Result Codes for an OTA Update

Result	Description
<code>RESULT_DEFAULT</code>	default state
<code>RESULT_SUCCESS</code>	update was successful
<code>RESULT_NO_STORAGE</code>	no storage available
<code>RESULT_OUT_OF_MEM</code>	out of memory
<code>RESULT_CONNECTION_LOST</code>	connection was lost
<code>RESULT_INTEGRITY_FAILED</code>	integrity check failed
<code>RESULT_UNSUP_FW</code>	unsupported firmware
<code>RESULT_INVALID_URI</code>	invalid uri provided
<code>RESULT_UPDATE_FAILED</code>	update failed
<code>RESULT_UNSUP_PROTO</code>	unsupported protocol

APPLICATION GUIDE

This section explains according to best practises how to develop IoT applications with flownexus and Zephyr.

3.1 Endpoint Interactions

flownexus can interact with any endpoint via the LwM2M methods `Read`, `Write`, `Execute` and `Observe`.

3.1.1 Read

Note

Currently not supported

3.1.2 Write

See Chapter *Data Flow: Backend -> Endpoint* for more details.

3.1.3 Execute

Note

Currently not supported

3.1.4 Observe

See chapter *Events via Composite Observe* to see events are generated from composite Observe.

Note

Currently Observe interactions are handled in Leshan, check *LwM2M Observe*.

3.2 Events

IoT devices often trigger events. An event is a significant change in the state of a device or its environment. As an example, there could be a machine that is monitored by an IoT device using multiple sensors. An event is triggered by an unusual combination of sensor readings. It is important to aggregate several states into a single event, so an application in the cloud can analyze the event in detail to know the environmental conditions at that moment.

The database model defines the *Events* table that represents events generated by *endpoints*.

3.2.1 Events via Composite Observe

In order to group multiple resources together in an unambiguous way, LwM2M composite resources are used. Once a composite resource is updated in the IoT device (endpoint), Leshan generates a composite event with all values of the composite resource. Django deserializes the event, stores its individual resources in the database and creates a new event. The individual resources within the event may have individual timestamps, the event itself has a separate timestamp in addition.

Note

Currently Observe interactions are handled in Leshan at *LwM2M Observe*.

3.3 Queue Mode

When configuring a energy saving endpoint that uses LTE-M or NB-IoT, queue mode must be enabled in Zephyr so the device will sleep between data transmissions. The [Zephyr documentation](#) explains the LwM2M Client in Zephyr in detail.

Listing 1: Enable Queue Mode in Zephyr with a registration update interval of 10 minutes

```
CONFIG_LWM2M_QUEUE_MODE_ENABLED=y
CONFIG_LWM2M_QUEUE_MODE_UPTIME=20
CONFIG_LWM2M_RD_CLIENT_STOP_POLLING_AT_IDLE=y

# Default lifetime is 10 minutes
CONFIG_LWM2M_ENGINE_DEFAULT_LIFETIME=600
```

See chapter *Data Flow: Backend -> Endpoint* to know know more about how queue mode is implemented in flownexus.

3.3.1 eDRX Settings

eDRX (Extended Discontinuous Reception) is a feature that allows the device to sleep for longer periods of time. The device will wake up periodically to check downlink paging messages. If new messages are available, the device will stay awake to receive them. Otherwise, the device will go back to sleep. - [Nordic Devzone Article](#)

Setting the eDRX interval to a value significantly smaller than the timeout value of the LwM2M server would theoretically allow flownexus to reach endpoints at any time.

Warning

This approach has not yet been tested! The typical way is to configure endpoints to connect to the server within specific intervals. Inbetween those intervalls, the endpoint will sleep and can't be reached from the server.

3.4 Firmware Setup

3.4.1 LwM2M Version

The used Leshan version uses LwM2M version 1.1. Make sure to enable this version in the Zephyr LwM2M client accordingly.

Listing 2: Enable LwM2M version 1.1 in Zephyr

```
CONFIG_LWM2M_VERSION_1_1=y
```

3.4.2 LwM2M Transport Format

Listing 3: Enable CBOR Format

```
CONFIG_ZCBOR=y
CONFIG_ZCBOR_CANONICAL=y
```

3.4.3 LwM2M SenML Format

Note

Currently not supported in flownexus.

Listing 4: Enable LwM2M version 1.1 in Zephyr

```
# SenML CBOR - default
CONFIG_LWM2M_RW_SENML_CBOR_SUPPORT=y
```

BUILD AND DEPLOY

4.1 Build Setup

The build instructions in the documentation are tested for a native Linux Machine. For MacOS or Windows consider creating a docker container build. One of the developers uses the following *devcontainer.json* build environment:

```
{
  "name": "Ubuntu",
  "image": "mcr.microsoft.com/devcontainers/base:jammy",
  "features": {
    "ghcr.io/devcontainers/features/docker-in-docker:2": {},
    "ghcr.io/devcontainers/features/docker-outside-of-docker:1": {}
  },
  "runArgs": [
    "--cap-add=NET_ADMIN",
    "--cap-add=MKNOD",
    "--device=/dev/net/tun",
    "--sysctl=net.ipv6.conf.all.disable_ipv6=0",
    "--sysctl=net.ipv6.conf.default.disable_ipv6=0"
  ],
  "postCreateCommand": "apt-get update && apt-get install -y iproute2 && echo 'IPv6 is enabled.'",
  "remoteUser": "root"
}
```

Before you we start with any development here are a few things you should get configured:

- Get the Zephyr SDK downloaded and configured in your root directory. You can find the instructions [here](#).
- Setup a virtual environment for the project.

```
host:~$ sudo apt update && sudo apt upgrade
host:~$ sudo apt install python3-pip python3.10-venv
host:~$ python3.10 -m venv venv
host:~$ source venv/bin/activate
host:~$ pip install --upgrade pip && pip install west
host:~$ mkdir workspace && cd workspace
host:~/workspace$ west init -m https://github.com/flownexus-lwm2m/flownexus --mr main
host:~/workspace$ west update
```

4.2 Container Environment

Both components run in a container. The Leshan server is running in a `openjdk:17-slim` container and the Django server is running in a `python:3.11-slim` container. This allows for an easy and reproducible setup of the server.

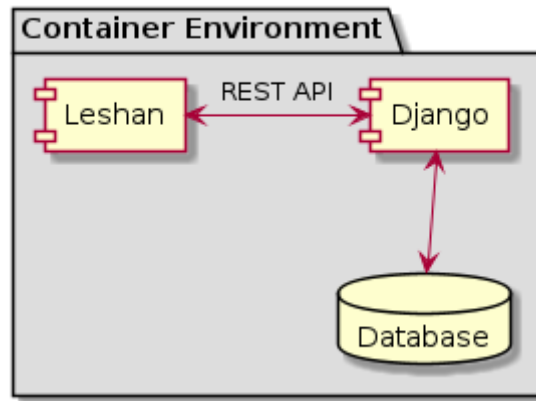


Fig. 1: Both components running in one machine using Podman Compose

The following diagram shows the Container Environment. The file `compose.yml` defines the services and their configuration. The file `Dockerfile.leshan` defines the Leshan container and the file `Dockerfile.django` defines the Django container.

Warning

Make sure to change the password to the admin console as well as other settings like `SECRET_KEY`, `DEBUG` flag in a production environment!

The container can be build and started with the following commands:

```

host:~/workspace/flownexus/server$ podman-compose build
host:~/workspace/flownexus/server$ podman-compose up

```

4.3 Setup a Virtual Server

flownexus can be deployed to a virtual server. This chapter explains a basic setup of a virtual server with a domain name. A requirement is to have a Linux server and a domain name. The domain name must point to the server, e.g. via a `A/AAAA-Record`.

The setup has been tested with a Debian 12 server with a 1C/1GB RAM configuration.

4.3.1 CA and self-signed Certificate

Leshan and the HTTPs download server for firmware binaries use self-signed certificates. The flownexus frontend uses certificates that have been issued via Let's Encrypt. The following commands create a self-signed certificate for the domain `flownexus.org`:

Create a Certificate Authority (CA)

1. Generate the CA Private Key:

```
openssl ecparam -genkey -name prime256v1 -out ca.key
```

2. Create a Self-Signed CA Certificate with 100 years validity:

```
openssl req -new -x509 -key ca.key -out ca.crt -days 36500 -subj "/CN=flownexus.  
→org"
```

Create a Server Certificate Signed by the CA

1. Generate the Server Private Key

```
openssl ecparam -genkey -name prime256v1 -out fw_flownexus_org.key
```

2. Generate a Certificate Signing Request (CSR)

```
openssl req -new -key fw_flownexus_org.key -out fw_flownexus_org.csr -subj "/"  
→CN=fw.flownexus.org"
```

3. Generate the Server Certificate Signed by the CA

```
openssl x509 -req -in fw_flownexus_org.csr -CA ca.crt -CAkey ca.key -  
→CAcreateserial -out fw_flownexus_org.crt -days 3650 -sha256
```

Generated Files

- ca.key: CA private key
- ca.crt: CA certificate
- fw_flownexus_org.key: Server private key
- fw_flownexus_org.csr: Server certificate signing request
- fw_flownexus_org.crt: Server certificate signed by the CA

Copy the server certificate and key to the server and store then in `/etc/nginx/ssl/`. Keep the CA certificate and key in a secure location.

4.3.2 Nginx as Reverse Proxy

The following steps show how to configure Nginx as a reverse proxy for the flownexus server. The Nginx server listens on port 443 and forwards the requests to the Django server running on port 8000:

Listing 1: Nginx setup, create Let's Encrypt certificate

```
# Update Sytem, install required packages and enable the firewall
vserver:~/ apt update
vserver:~/ apt install git podman podman-compose nginx certbot python3-certbot-nginx

# Generate a certificate with letsencrypt:
vserver:~/ certbot --nginx -d flownexus.org -d www.flownexus.org
vserver:~/ Create nginx config at /etc/nginx/sites-available/flownexus (see example_
→below)
```

Listing 2: Nginx config for the Frontend `/etc/nginx/sites-available/flownexus.org`

```
1 server {
2     listen 443 ssl http2;
3     listen [::]:443 ssl http2;
4     server_name flownexus.org www.flownexus.org;
```

(continues on next page)

(continued from previous page)

```

5      error_log /var/log/nginx/flownexus.org.error.log;
6      access_log /var/log/nginx/flownexus.org.access.log;
7      ssl_certificate /etc/letsencrypt/live/flownexus.org/fullchain.pem; # managed
↳ by Certbot
8
9      ssl_certificate_key /etc/letsencrypt/live/flownexus.org/privkey.pem; #
↳ managed by Certbot
10
11      location / {
12          proxy_pass http://127.0.0.1:8000/;
13          proxy_set_header Host $http_host;
14          proxy_set_header X-Real-IP $remote_addr;
15          proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
16          proxy_set_header X-Forwarded-Proto $scheme;
17          proxy_set_header X-Frame-Options SAMEORIGIN;
18      }
19 }
20
21 server {
22     listen 80;
23     listen [::]:80;
24     server_name flownexus.org www.flownexus.org;
25
26     # Redirect all HTTP requests to HTTPS
27     return 301 https://$host$request_uri;
28 }

```

Listing 3: Nginx config for the https dl Server /etc/nginx/sites-available/fw.flownexus.org

```

1 server {
2     listen 443 ssl;
3     listen [::]:443 ssl http2;
4     server_name fw.flownexus.org;
5
6     ssl_certificate /etc/nginx/ssl/fw_flownexus_org.crt;
7     ssl_certificate_key /etc/nginx/ssl/fw_flownexus_org.key;
8
9     location /binaries {
10         root /var/www/flownexus/;
11         # Debug option: uncomment to list files
12         # autoindex on;
13     }
14 }
15
16 server {
17     listen 80;
18     server_name fw.flownexus.org;
19
20     # Redirect all HTTP requests to HTTPS
21     return 301 https://$host$request_uri;
22 }

```

After creating the Nginx config, activate the config and restart the Nginx.

Listing 4: Activate the Nginx config

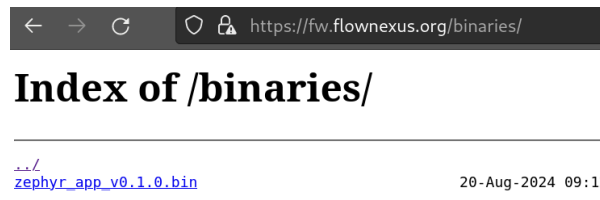
```
# Activate the Nginx config:
vserver:~/ ln -s /etc/nginx/sites-available/flownexus.org /etc/nginx/sites-enabled/
vserver:~/ ln -s /etc/nginx/sites-available/fw.flownexus.org /etc/nginx/sites-enabled/

# Test the Nginx config:
vserver:~/ nginx -t

# Restart Nginx:
vserver:~/ systemctl restart nginx
```

4.3.3 Test the Download Server

If you have setup an A/AAAA-Record, you can now test the download server. It is available at <https://fw.flownexus.org/binaries>. If you uncomment the option `autoindex on`; in the Nginx config, you can list the files in the directory.



4.3.4 Start flownexus

After the setup, download flownexus and start it with using podman-compose in detached mode. Make sure to change the `DEPLOY_SECRET_KEY` and `DEBUG` flag in the `settings.py` file before deploying.:

Listing 5: Start flownexus with podman-compose

```
vserver:~/ git clone https://github.com/flownexus-lwm2m/flownexus.git
# Change the DEPLOY_SECRET_KEY and DEBUG flag in the settings.py file
vserver:~/flownexus/server$ podman-compose up -d
```

flownexus is now available at <https://flownexus.org>. The server is running in a Docker container and the Nginx server is used as a reverse proxy.

4.3.5 Security Considerations

Consider enabling the firewall and only keep required ports open:

- Port 80, TCP: HTTP
- Port 443, TCP: HTTPS
- Port 22, TCP: SSH
- Port 5683, UDP: CoAP

Warning

flownexus is not production ready. This server setup is only intended for testing purposes.

The current flownexus configuration uses the default Django `DEPLOY_SECRET_KEY` and enables the `DEBUG` flag. This is a security risk and must be change before deploying.

Currently, the default django inbuild webserver is used. This is not recommended for production use. Consider using a production-ready webserver like Nginx or Apache.

SIMULATION

flownexus provides a modular simulation platform located in the `simulation/` directory. This platform allows you to simulate IoT devices at different levels of fidelity, from lightweight API mocks to full-stack Zephyr firmware simulations.

5.1 Simulation Backends

The platform supports two primary backends:

1. **Mock Backend:** A lightweight Python-based simulator that interacts directly with the Django Ingestion API. It is ideal for frontend development and load testing.
2. **Zephyr Backend:** A high-fidelity simulator that builds and runs actual Zephyr OS binaries (`native_sim`) inside a containerized network environment.

5.2 Simulation Dispatcher (`simulate.py`)

The `simulate.py` script is the central entry point for all simulations. It uses YAML configuration files to define the simulation behavior.

Listing 1: Usage of `simulate.py`

```
host:~/flownexus/simulation$ python3 simulate.py --help
usage: simulate.py [-h] --config CONFIG [--build] [--run]
                  [--type {mock,zephyr}] [--count DEVICE_COUNT]

Flownexus Device Simulation Dispatcher

options:
  -h, --help            show this help message and exit
  --config CONFIG        Path to simulation YAML config file (Required)
  --build                Build binaries (Zephyr only)
  --run                 Run simulation
  --type {mock,zephyr}  Override simulation type
  --count DEVICE_COUNT  Override number of devices
```

5.3 Mock Simulation

The Mock backend is designed for rapid development of the flownexus frontend. It bypasses the complexity of LwM2M and Zephyr, sending telemetry data directly to the server.

5.3.1 Features

- **Sine-wave Data:** Generates smooth temperature and humidity curves, making it easy to verify frontend charts.
- **Zero Dependencies:** Does not require Docker, Podman, or Zephyr toolchains.
- **Fast:** Instant startup and low resource overhead.

5.3.2 Running the Mock Simulation

The easiest way to start a mock simulation is via the project Makefile:

```
host:~/flownexus$ make run-mock
```

Alternatively, you can use the provided `sim_mock.yaml` configuration with the script directly:

```
host:~/flownexus/simulation$ python3 simulate.py --config sim_mock.yaml
```

Note

Ensure your Django server is running before starting the simulation. By default, it targets `http://localhost:8000/flownexus/ingest`.

5.4 Zephyr Simulation

The Zephyr backend runs actual firmware binaries. This is used for end-to-end (E2E) testing and verifying LwM2M protocol compliance.

5.4.1 Prerequisites

The Zephyr simulation requires a working Podman installation and the Python `docker` and `PyYAML` libraries.

```
host:~$ apt install podman podman-compose
host:~$ pip install docker PyYAML
```

Furthermore, the flownexus server stack should be running. The script expects the network `server_mynetwork` to be available.

5.4.2 Running the Zephyr Simulation

The easiest way to build and run the Zephyr simulation is via the project Makefile:

```
# Build the binaries (Required once or after code changes)
host:~/flownexus$ make build-sim

# Run the E2E test suite
host:~/flownexus$ make test-e2e
```

5.4.3 Manual Control

You can also use `simulate.py` directly with the Zephyr configuration:

```
# Build 5 devices
host:~/flownexus/simulation$ python3 simulate.py --config sim_zephyr.yaml --build --
↪count 5
```

(continues on next page)

(continued from previous page)

```
# Run the simulation and connect to the local server
host:~/flownexus/simulation$ python3 simulate.py --config sim_zephyr.yaml --run --
↪local
```

5.5 Configuration (YAML)

Simulations are configured via YAML files. This allows you to check in specific test scenarios into version control.

Listing 2: Example sim_mock.yaml

```
type: mock
url: "http://localhost:8000/flownexus/ingest"
device_count: 5
interval: 2.0 # Seconds between updates
duration: 0 # 0 = Run forever
run: true # Automatically start simulation
```

Listing 3: Example sim_zephyr.yaml

```
type: zephyr
device_count: 1
local_leshan: true # Connect to the Leshan container
verbose: false
delay: 0 # Startup delay between instances (ms)
build: false
run: false
```

5.6 Integration Testing

The mock simulation is integrated into the Django test suite. You can run automated ingestion tests using:

```
host:~/flownexus/server/django$ python manage.py test sensordata.tests.test_mock_
↪simulation
```

This test uses Django's `LiveServerTestCase` to spin up a real HTTP server and verify that the simulation backend can successfully register devices and ingest data.

5.7 External Resources

See also

- Zephyr LwM2M API
- Zephyr LwM2M Client Sample
- Zephyr Native Sim Board
- Zephyr Networking with Multiple Instances

DOCUMENTATION

6.1 Build the documentation

```
host:~$ sudo apt-get install default-jre plantuml graphviz
host:~$ source venv/bin/activate
host:~$ cd workspace/ flownexus/doc
host:~/workspace/ flownexus/doc$ pip install -r requirements.txt
host:~/workspace/ flownexus/doc$ tox -e html
```

Open the generated index.html in the doc/build directory in your browser.

GLOSSARY

Terms are usually explained and indexed directly in the documentation chapters. Additionally, some general terms are defined here to provide a quick reference. The terms defined here, as well as the indexed terms from the documentation chapters can be referenced in the documentation by using the `:term:` role.

7.1 Additional Terms and Definitions

LwM2M

Lightweight Machine to Machine protocol.

Leshan

LwM2M server implementation as one main component of the system.

Backend

Django instance that is responsible for the REST API, database and visualization.

Frontend

Web application that visualizes the data from the backend, integrated into Django.

Object

A logical grouping of one or multiple Resources in LwM2M. An Object is identified by an Object ID.

INTERNAL DJANGO

The API Documentation is available at <https://flownexus-lwm2m.github.io/flownexus/>.

WEBLOG

9.1 Release v0.1.0

#3

Date

Aug 4, 2024

Author

<https://github.com/jonas-rem>

Tags

Release Notes

First release of the project. This is a development release.

- Created a flownexus logo.
- Generic database model for LwM2M objects.
- Added a frontend mockup based on AtminLTE.
- ReST APIs inbetween Leshan and Django.
- Asynchronous task queue for Endpoint interactions from Django via Celery.
- Bidirectional communication between L2M2M endpoints and Django via Leshan.
- Subscribe to single and composite Observations in Leshan statically.
- OTA Update functionality on the server side. Full-functioning firmware example pending.

9.2 Update Documentation & Add Weblog

#2

Date

May 15, 2024

Author

<https://github.com/Kappuccino111>

Tags

Add, Update

- Switched the new documentation to *Sphinx-Book-Theme* for better readability and navigation.
- Added a new section for the weblog to keep track of updates and changes in the project.

9.3 Project Starts

#1

Date

January 1, 2024

Author

<https://github.com/jonas-rem>

Tags

Update

- The project has started and the initial documentation has been added.

INDEX

B

Backend, [25](#)

E

Endpoint, [5](#)

EndpointOperation, [6](#)

Event, [6](#)

EventResource, [6](#)

F

Firmware, [6](#)

FirmwareUpdate, [6](#)

Frontend, [25](#)

L

Leshan, [25](#)

LwM2M, [25](#)

LwM2M observe, [7](#)

O

Object, [25](#)

R

Resource, [6](#)

ResourceType, [5](#)

S

Single and CompositeObserve, [7](#)